

```

import React, { useEffect, useRef, useCallback } from "react";
import { X, Check, AlertTriangle } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// ✂ LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// ① LOCAL TYPE ISOLATION GATEWAY
export interface ConfirmationBoxProps {
  open: boolean;
  onClose: () => void;
  onConfirm: () => void;
  title: string;
  description: string;
  isDestructive?: boolean;
  confirmLabel?: string;
  cancelLabel?: string;
}

export function ConfirmationBox({
  open,
  onClose,
  onConfirm,
  title,
  description,
  isDestructive = false,
  confirmLabel = "Confirm",
  cancelLabel = "Cancel",
}: ConfirmationBoxProps) {
  const dialogRef = useRef<HTMLDialogElement>(null);
  const cancelButtonRef = useRef<HTMLButtonElement>(null);
  const confirmButtonRef = useRef<HTMLButtonElement>(null);
  const previousActiveElement = useRef<HTMLElement | null>(null);

  const handleKeyDown = useCallback(
    (event: KeyboardEvent) => {
      if (event.key === "Escape") {
        event.preventDefault();
        onClose();
      }
    },
    [onClose],
  );

  const handleFocusTrap = useCallback((event: KeyboardEvent) => {
    if (event.key !== "Tab" || !dialogRef.current) return;
  });

```

```

const focusableElements = dialogRef.current.querySelectorAll<HTMLElement>(
  'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
);

const firstElement = focusableElements[0];
const lastElement = focusableElements[focusableElements.length - 1];

if (event.shiftKey && document.activeElement === firstElement) {
  event.preventDefault();
  lastElement?.focus();
} else if (!event.shiftKey && document.activeElement === lastElement) {
  event.preventDefault();
  firstElement?.focus();
}
}, []);

const handleConfirm = useCallback(() => {
  onConfirm();
  onClose();
}, [onConfirm, onClose]);

useEffect(() => {
  if (open) {
    previousActiveElement.current = document.activeElement as HTMLElement;
    document.addEventListener("keydown", handleKeyDown);
    document.addEventListener("keydown", handleFocusTrap);
    document.body.style.overflow = "hidden";

    dialogRef.current?.showModal();

    // CRITICAL: Focus Cancel button by default to prevent accidental Enter key
confirmation
    cancelButtonRef.current?.focus();

    return () => {
      document.removeEventListener("keydown", handleKeyDown);
      document.removeEventListener("keydown", handleFocusTrap);
      document.body.style.overflow = "";
      previousActiveElement.current?.focus();
      dialogRef.current?.close();
    };
  }
}, [open, handleKeyDown, handleFocusTrap]);

if (!open) return null;

const confirmBgColor = isDestructive ? "bg-destructive" : "bg-brand-primary";
const confirmHoverColor = isDestructive
  ? "hover:bg-destructive/90"
  : "hover:bg-brand-hover";

```

```

const confirmActiveColor = isDestructive
  ? "active:bg-destructive"
  : "active:bg-brand-primary";
const confirmRingColor = isDestructive
  ? "focus-visible:ring-destructive"
  : "focus-visible:ring-brand-primary";
const iconColor = isDestructive ? "text-destructive" : "text-brand-primary";
const bgColor = isDestructive ? "bg-destructive/10" : "bg-brand-primary/10";
const borderColor = isDestructive
  ? "border-destructive/30"
  : "border-brand-primary/30";

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
    <div
      className={cn(
        "fixed inset-0 z-40",
        "bg-black/50 backdrop-blur-sm",
        "transition-opacity duration-300 ease-out",
        "opacity-100",
      )}
      onClick={onClose}
      aria-hidden="true"
    />
    <dialog
      ref={dialogRef}
      className={cn(
        "relative z-50 w-full max-w-md",
        "bg-card/95 backdrop-blur-[var(--backdrop-blur)]",
        "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
        "rounded-[var(--radius-surface)]",
        "shadow-[var(--shadow-xl)]",
        "overflow-hidden",
        "transition-all duration-200 ease-out",
        open
          ? "opacity-100 scale-100"
          : "opacity-0 scale-95 pointer-events-none",
      )}
      role="alertdialog"
      aria-modal="true"
      aria-labelledby="confirmation-box-title"
      aria-describedby="confirmation-box-description"
    >
      <div className={cn("p-6", bgColor, borderColor, "border-t-4")}>
        <div className="flex items-start gap-4">
          <div
            className={cn(
              "flex-shrink-0 w-10 h-10 rounded-full flex items-center
justify-center",
              bgColor,

```

```

    })
    aria-hidden="true"
  >
    <AlertTriangle
      className={cn("w-5 h-5", iconColor)}
      strokeWidth={2.5}
    />
  </div>
  <div className="flex-1 min-w-0">
    <h2
      id="confirmation-box-title"
      className="text-lg font-semibold text-foreground leading-tight"
    >
      {title}
    </h2>
    <p
      id="confirmation-box-description"
      className="mt-2 text-sm text-muted-foreground leading-relaxed"
    >
      {description}
    </p>
  </div>
</div>
<div className="flex items-center justify-end gap-3 p-4 border-t
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]">
  <button
    ref={cancelButtonRef}
    type="button"
    onClick={onClose}
    className={cn(
      "inline-flex items-center justify-center gap-2",
      "h-10 px-4 py-2 text-base font-medium",
      "rounded-[var(--radius-surface)]",
      "bg-secondary text-secondary-foreground",
      "hover:bg-secondary/80",
      "active:scale-95 active:bg-secondary",
      "transition-all duration-150 ease-out",
      "focus-visible:outline-none",
      "focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      "disabled:pointer-events-none disabled:opacity-50
disabled:cursor-not-allowed",
    )}
  >
    <X className="w-4 h-4" aria-hidden="true" />
    {cancelLabel}
  </button>
  <button
    ref={confirmButtonRef}

```

```

    type="button"
    onClick={handleConfirm}
    className={cn(
      "inline-flex items-center justify-center gap-2",
      "h-10 px-4 py-2 text-base font-medium",
      "rounded-[var(--radius-surface)]",
      confirmBgColor,
      "text-white",
      confirmHoverColor,
      "active:scale-95",
      confirmActiveColor,
      "transition-all duration-150 ease-out",
      "focus-visible:outline-none",
      confirmRingColor,
      "focus-visible:ring-2 focus-visible:ring-offset-2",
      "focus-visible:ring-offset-background",
      "disabled:pointer-events-none disabled:opacity-50",
      "disabled:cursor-not-allowed",
    )}
  >
    <Check className="w-4 h-4" aria-hidden="true" />
    {confirmLabel}
  </button>
</div>
</dialog>
</div>
);
}

export default ConfirmationBox;

```